

Algoritmi utili in crittografia

Fabio Stumbo

Università di Ferrara

L'algoritmo euclideo

L'**algoritmo euclideo** è l'algoritmo alla base di tutti i calcoli algebrici.

Esso permette di calcolare il massimo comun divisore di due numeri interi e, inoltre, di esprimerlo come combinazione dei due numeri dati (**identità di Bézout**)

Ricordiamo il teorema di Euclide che dà origine all'algoritmo.

Teorema 1. *Siano r_0, r_1 numeri interi con $r_1 \neq 0$ (per semplicità, possiamo supporre che siano entrambi positivi) e siano q_2, r_2 numeri interi tali che $r_0 = q_2 r_1 + r_2$ con $0 \leq r_2 < r_1$. Allora*

$$\text{MCD}(r_0, r_1) = \text{MCD}(r_1, r_2).$$

Dimostrazione. Se d è un divisore comune di r_0 ed r_1 , da $r_2 = r_0 - r_1 q_2$ si ottiene subito $d \mid r_2$.

Se d è un divisore comune di r_1 ed r_2 , da $r_0 = r_1 q + r_2$ si ottiene subito $d \mid r_0$.

Tutti i divisori comuni di r_0, r_1 coincidono quindi con i divisori comuni di r_1, r_2 pertanto, in particolare, i due massimi divisori comuni coincidono. $\square$

Applicando in modo induttivo il Teorema si ha una successione $r_0, r_1, r_2, r_3, \dots, r_i$ che è strettamente decrescente (con l'eventuale eccezione dei primi due termini), cioè tale che

$$r_1 > r_2 > r_3 > \dots > r_i.$$

Dato che sono tutti numeri non negativi, deve esistere n tale che $r_{n+1} > r_{n+2} = 0$.

Poiché per ogni intero a vale $\text{MCD}(a, 0) = a$, si ha

$$\text{MCD}(r_0, r_1) = \dots = \text{MCD}(r_{n+1}, r_{n+2}) = \text{MCD}(r_{n+1}, 0) = r_{n+1}$$

e quindi, in tal modo, si calcola $\text{MCD}(r_0, r_1)$.

Esempio 2. Eseguiamo l'algoritmo euclideo secondo lo schema delle equazioni 4 per i numeri 123456 e 56789. Per meglio evidenziare i resti ed il loro ruolo, scriviamo i numeri di partenza in *corsivo* ed i resti successivi in **grassetto**.

Partendo da queste operazioni, è possibile esprimere il massimo comun divisore come combinazione lineare dei due numeri di partenza

$$\begin{array}{ll} 123456 = 2 \cdot 56789 + \mathbf{9878} & \mathbf{1} = \mathbf{9} - 4 \cdot \mathbf{2} \\ 56789 = 5 \cdot \mathbf{9878} + \mathbf{7399} & = \mathbf{9} - 4 \cdot (\mathbf{38} - 4 \cdot \mathbf{9}) \\ \mathbf{9878} = 1 \cdot \mathbf{7399} + \mathbf{2479} & = -4 \cdot \mathbf{38} + 17 \cdot \mathbf{9} \\ \mathbf{7399} = 2 \cdot \mathbf{2479} + \mathbf{2441} & = -4 \cdot \mathbf{38} + 17 \cdot (\mathbf{2441} - 64 \cdot \mathbf{38}) \end{array}$$

$$\begin{aligned}
2479 &= 1 \cdot 2441 + 38 & &= 17 \cdot 2441 - 1092 \cdot 38 \\
2441 &= 64 \cdot 38 + 9 & &= 17 \cdot 2441 - 1092 \cdot (2479 - 1 \cdot 2441) \\
38 &= 4 \cdot 9 + 2 & &= -1092 \cdot 2479 + 1109 \cdot 2441 \\
9 &= 4 \cdot 2 + 1 & &= -1092 \cdot 2479 + 1109 \cdot (7399 - 2 \cdot 2479) \\
2 &= 2 \cdot 1 & &= 1109 \cdot 7399 - 3310 \cdot 2479 \\
& & &= 1109 \cdot 7399 - 3310 \cdot (9878 - 1 \cdot 7399) \\
& & &= -3310 \cdot 9878 + 4419 \cdot 7399 \\
& & &= -3310 \cdot 9978 + 4419 \cdot (56789 - 5 \cdot 9878) \\
& & &= 4419 \cdot 56789 - 25405 \cdot 9878 \\
& & &= 4419 \cdot 56789 - 25405 \cdot (123456 - 2 \cdot 56789) \\
& & &= -25405 \cdot 123456 + 55229 \cdot 56789
\end{aligned}$$

△

Si ottiene così l'**identità di Bézout**:

$$1 = -25405 \cdot 123456 + 55229 \cdot 56789$$

L'identità di Bézout può essere calcolata più efficientemente “in parallelo” allo svolgimento delle operazioni dell'algoritmo euclideo.

Teorema 3 (Algoritmo Euclideo Esteso). *Siano a, b due numeri interi non nulli. Definiamo*

$$\begin{cases} r_0 = a \\ r_1 = b \end{cases}, \quad \begin{cases} x_0 = 1 \\ x_1 = 0 \end{cases}, \quad \begin{cases} y_0 = 0 \\ y_1 = 1 \end{cases}. \quad (1)$$

e, per induzione, definiamo per $i \geq 0$, $q_{i+2}, r_{i+2} \in A$ in modo tale che

$$r_i = q_{i+2}r_{i+1} + r_{i+2} \quad \text{con } 0 \leq r_{i+2} < r_{i+1} \quad (2)$$

Inoltre, sempre per induzione, definiamo

$$\begin{cases} x_{i+2} = x_i - q_{i+2}x_{i+1} \\ y_{i+2} = y_i - q_{i+2}y_{i+1} \end{cases} \quad (3)$$

Allora la successione dei resti r_i delle divisioni euclidee

$$\begin{aligned}
a &= q_2b + r_2 \\
b &= q_3r_2 + r_3 \\
r_2 &= q_4r_3 + r_4 \\
&\vdots \\
r_{n-1} &= q_{n+1}r_n + r_{n+1} \\
r_n &= q_{n+2}r_{n+1}
\end{aligned} \quad (4)$$

arriva a 0 (cioè esiste n tale che $r_{n+2} = 0$) in un numero finito di passi e, per $i = 0, \dots, n+1$, si ha che

$$1. \text{ MCD}(a, b) = \text{MCD}(r_i, r_{i+1}) = r_{n+1};$$

$$2. \ r_i = x_i a + y_i b;$$

In particolare, per $i = n + 1$ si ha l'identità di Bézout:

$$\text{MCD}(a, b) = x_{n+1}a + y_{n+1}b. \quad (5)$$

Dimostrazione. Il punto 1. l'abbiamo già visto. Il punto 2. si dimostra facilmente per induzione. $\square$

Esempio 4. Ricalcoliamo $\text{MCD}(123456, 56789)$ secondo la procedura indicata nel Teorema. $\triangle$

i	q_i	r_i	x_i	y_i
0	–	123456	1	0
1	–	56789	0	1
2	2	9878	1	–2
3	5	7399	–5	11
4	1	2479	6	–13
5	2	2441	–17	37
6	1	38	23	–50
7	64	9	–1489	3237
8	4	2	5979	–12998
9	4	1	–25405	55229
10	2	0	56789	–123456

Questa procedura è più efficiente perché non richiede di ricordare tutte le divisioni ma occupa solo 7 variabili.

Osservazione 5. Si può dimostrare che l'algoritmo euclideo permette di calcolare $\text{MCD}(a, b)$ con un numero di operazioni che è *polinomiale* (di esponente 2) nel numero delle cifre di a, b , mentre se si volesse calcolare facendo prima la fattorizzazione dei numeri a, b il costo computazionale sarebbe *enormemente* più alto: in altre parole, si tratta di un algoritmo molto efficiente. $\triangle$

Il Teorema cinese del resto

Teorema 6 (Teorema cinese del resto). *Siano m, n due numeri interi non nulli. Il sistema di congruenze*

$$\begin{cases} x \equiv a \pmod{m} \\ x \equiv b \pmod{n} \end{cases} \quad (6)$$

è sempre risolubile per ogni coppia a, b se, e solo se, $\text{MCD}(m, n) = 1$.

In tal caso, siano $1 = \text{MCD}(m, n) = \lambda m + \mu n$ e $k = a - b$. Allora le soluzioni del sistema sono tutti e soli gli elementi della classe di congruenza $x \equiv a - k\lambda m \pmod{mn}$.

Dimostrazione. Supponiamo $d = \text{MCD}(m, n) > 1$ e consideriamo il sistema

$$\begin{cases} x \equiv 1 \pmod{m} \\ x \equiv 0 \pmod{n} \end{cases}$$

Se x è una soluzione del sistema, allora

$$\begin{cases} x = 1 + um = 1 + um'd \\ x = vn = vn'd \end{cases}$$

da cui $1 + um'd = vn'd$ che implica $d|1$, assurdo.

Se invece $1 = \text{MCD}(m, n) = \lambda m + \mu n$, poniamo $x = a - k\lambda m$. Per definizione, $x \equiv a \pmod{m}$. Inoltre,

$$x = a - k\lambda m = a - k(1 - \mu n) = a - k + k\mu n = a - (a - b) + k\mu n = b + k\mu n$$

e quindi $x \equiv b \pmod{n}$.

Se x' è un'altra soluzione del sistema allora $m|x - x'$ e $n|x - x'$, da cui $mn = \text{mcm}(m, n) | x - x'$. $\square$

Osservazione 7. 1. L'identità di Bézout implica $1 \equiv \lambda m \pmod{n}$, cioè $\lambda = m^{-1} \pmod{n}$.

2. Se si ha un sistema di più di due congruenze, si risolve il sistema formato dalle prime due congruenze; la soluzione di questo sistema è una congruenza, pertanto sostituendo le prime due congruenze con la congruenza trovata si ottiene un sistema con una congruenza in meno: si procede poi ricorsivamente. $\triangle$

Esempio 8. Risolviamo il sistema

$$\begin{cases} x \equiv 315 \pmod{123456} \\ x \equiv 432 \pmod{56789} \end{cases}$$

Si ha

$$1 = -25405 \cdot 123456 + 55229 \cdot 56789$$

quindi, nella notazione del TCdR,

$$\begin{aligned} m &= 123456 \\ n &= 56789 \\ a &= 315 \\ b &= 432 \\ \lambda &= -25405 \\ \mu &= 55229 \\ k &= a - b = -117 \end{aligned}$$

e quindi la soluzione del sistema è

$$\begin{aligned} x &\equiv a - k\lambda m \pmod{mn} \\ &\equiv 315 - (-117) \cdot (-25405) \cdot 123456 \pmod{123456 \cdot 56789} \\ &\equiv -366958762245 \pmod{7010942784} \\ &\equiv 4621205307 \pmod{7010942784} \end{aligned}$$

$\triangle$

L'algoritmo “*square and multiply*”

L'algoritmo *square&multiply* permette di calcolare efficientemente potenze grandi in aritmetica modulare. Esso si basa sulla rappresentazione in base 2 di un numero.

Supponiamo di voler calcolare

$$a^n \pmod{m}.$$

Scriviamo n in base 2:

$$n = n_i 2^i + n_{i-1} 2^{i-1} + \dots + n_2 2^2 + n_1 2 + n_0$$

con $n_j \in \{0, 1\}$.

Da

$$n = 2 \left(2 \left(2 \left(\dots 2 \left(2(2n_i + n_{i-1}) + n_{i-2} \right) + \dots + n_3 \right) + n_2 \right) + n_1 \right) + n_0$$

otteniamo

$$a^n = \left(\left(\left((a^{n_i})^2 a^{n_{i-1}} \right)^2 \dots a^{n_2} \right)^2 a^{n_1} \right)^2 a^{n_0}.$$

Questo vuol dire che possiamo calcolare a^n tramite una serie di i quadrati e, al più, $i+1$ prodotti per a .

La procedura, nel dettaglio, è la seguente:

1. si considera la stringa $n_i n_{i-1} \dots n_1 n_0$ delle cifre binarie di n ;
2. si mette un simbolo Q tra due cifre successive: $n_i Q n_{i-1} Q \dots Q n_1 Q n_0$.
3. partendo da $\alpha_0 = 1$, si calcola α_{j+1} secondo la regola

$$\alpha_{j+1} = \begin{cases} \alpha_j \cdot a^{n_{i-j}} & \text{se } 2 \mid j \\ \alpha_j^2 & \text{se } 2 \nmid j \end{cases}$$

Si ha $\alpha_{2i+1} = a^n$.

Osservazione 9. La regola 3. vuol dire semplicemente: “*si inizializza $\alpha = 1$ e si scorre la stringa $n_i Q n_{i-1} Q \dots Q n_1 Q n_0$ da sinistra a destra facendo le seguenti operazioni:*

- ▷ *se si incontra un 1, si moltiplica α per a ;*
- ▷ *se si incontra uno 0, si lascia α invariato;*
- ▷ *se si incontra una Q , si fa il quadrato di α ”.*

Alla fine in α sarà registrato il valore a^n . △

Esempio 10. Calcoliamo $12^{100} \pmod{34}$. Si ha $100 = 64 + 32 + 4 = 2^6 + 2^5 + 2^2 = 1100100_2$ per cui la stringa del punto 2. è: $1Q1Q0Q0Q1Q0Q0$.

Dato che quando si incontra uno 0 il numero non cambia, possiamo semplificare la stringa eliminando gli zeri e considerare la stringa che dà le operazioni effettive da fare: $XQXQQQXQQ$, dove la X significa “*moltiplicare per a* ” mentre la Q significa “*fare il quadrato*” (da qui il nome dell'algoritmo).

Il calcolo da fare è quindi

$$\overline{1} \xrightarrow{X} \overline{12} \xrightarrow{Q} \overline{8} \xrightarrow{X} \overline{28} \xrightarrow{Q} \overline{2} \xrightarrow{Q} \overline{4} \xrightarrow{Q} \overline{16} \xrightarrow{X} \overline{22} \xrightarrow{Q} \overline{8} \xrightarrow{Q} \overline{30}.$$

Infatti,

$$1 \xrightarrow{X} a \xrightarrow{Q} a^2 \xrightarrow{X} a^3 \xrightarrow{Q} a^6 \xrightarrow{Q} a^{12} \xrightarrow{Q} a^{24} \xrightarrow{X} a^{25} \xrightarrow{Q} a^{50} \xrightarrow{Q} a^{100}$$

o, in notazione binaria,

$$\begin{aligned} 1 &\xrightarrow{X} a^1 \xrightarrow{Q} a^{10} \xrightarrow{X} a^{11} \xrightarrow{Q} a^{110} \xrightarrow{Q} a^{1100} \xrightarrow{Q} a^{11000} \xrightarrow{X} \\ &a^{11001} \xrightarrow{Q} a^{110010} \xrightarrow{Q} a^{1100100}. \end{aligned}$$

△

Osservazione 11. Dato che $i = \lfloor \log_2 n \rfloor + 1$, il numero di operazioni così richieste è per calcolare $a^n \bmod m$ è *polinomiale (di grado 1)* nel numero delle cifre di n , mentre se si facesse il calcolo diretto sarebbe *esponenziale* nel numero delle cifre: si tratta quindi di un algoritmo molto efficiente. △

Pseudocodice

In questa sezione proponiamo lo pseudocodice necessario per implementare i tre algoritmi visti.

Algoritmo Euclideo Esteso

Algoritmo 1: ALGORITMO EUCLIDEO, VERSIONE ESTESA(a, b)

$u \leftarrow a, \quad x \leftarrow 1, \quad y \leftarrow 0$

$v \leftarrow b, \quad x' \leftarrow 0, \quad y' \leftarrow 1$

if $v = 0$

then return $(u; x, y)$

while $v \neq 0$

$\left\{ \begin{array}{l} q \leftarrow \lfloor \frac{u}{v} \rfloor \\ r \leftarrow u - vq \\ u \leftarrow v \\ v \leftarrow r \\ \text{do } \left\{ \begin{array}{l} temp \leftarrow x - qx' \\ x \leftarrow x' \\ x' \leftarrow temp \\ temp \leftarrow y - qy' \\ y \leftarrow y' \\ y' \leftarrow temp \end{array} \right. \end{array} \right.$

return $(u; x, y)$

osservazione: $u = \text{MCD}(a, b)$ e $u = xm + yb$

Teorema Cinese del Resto

Algoritmo 2: SISTEMA DI CONGRUENZE($a, b; m, n$)

$(d; \lambda, \mu) \leftarrow \text{MCD}(m, n)$

osservazione: $d = \lambda m + \mu n$ cfr. AEE

if $d \neq 1$

then return (I numeri m, n non sono coprimi)

else $\left\{ \begin{array}{l} k \leftarrow (a - b) \\ m' \leftarrow m/d \\ n' \leftarrow n/d \\ M \leftarrow \text{mcm}(m, n) \\ \text{osservazione: } M = mn \\ \text{return } (a - k\lambda m + (M)) \end{array} \right.$

Osservazione 12. Come abbiamo visto nell'esempio svolto, non è detto che $a - k\lambda m$ sia una classe di resto "canonica" modulo M , cioè non è detto che si abbia $0 \leq a - k\lambda m < mn$. Se questo è il caso, si può estendere il codice in modo che restituisca una classe di resto canonica. $\triangle$

Square & Multiply

Algoritmo 3: SQUARE AND MULTIPLY (a, n, m)

osservazione: Calcola $a^n \bmod m$

$p \leftarrow a \bmod m$

$i \leftarrow \lfloor \log_2 n \rfloor$

$exp \leftarrow n - 2^i$

for $j = i - 1$ **downto** 0

do $\left\{ \begin{array}{l} p \leftarrow p^2 \bmod m \\ n_j \leftarrow \lfloor \frac{exp}{2^j} \rfloor \\ \text{if } n_j = 1 \\ \quad \text{then } p \leftarrow pa \bmod m \\ exp \leftarrow exp - n_j 2^j \end{array} \right.$

return (p)